

Informe Proyecto 1: Calculadora de 4 bits

Arquitectura de Computadores · Semestre 2026-2 Universidad de los Andes — Facultad de Ingeniería y Ciencias Aplicadas Profesor: Jorge Gómez Mir

Integrantes: Lucas Fernández · Maximiliano Ferrando · Renato Rivadeneira · Matías Ureta

Repositorio: github.com/Max-Ferrando/Proyecto-1-Arquitectura-

1. OBJETIVO Y ALCANCE

Se diseñó e implementó una calculadora de 4 bits en complemento a dos (rango -8 a $+7$) sobre una FPGA Lattice iCE40 HX1K de la Nandland Go Board. La calculadora soporta seis operaciones seleccionables mediante un código de 3 bits y almacena el resultado en un registro, de modo que puede encadenarse como operando de la operación siguiente.

La restricción central del proyecto es que toda la lógica combinacional debe describirse exclusivamente con compuertas lógicas, sin usar los operadores de alto nivel de Verilog. Esta restricción condiciona todas las decisiones de diseño que se explican a continuación.

2. DISEÑO GENERAL Y ARQUITECTURA

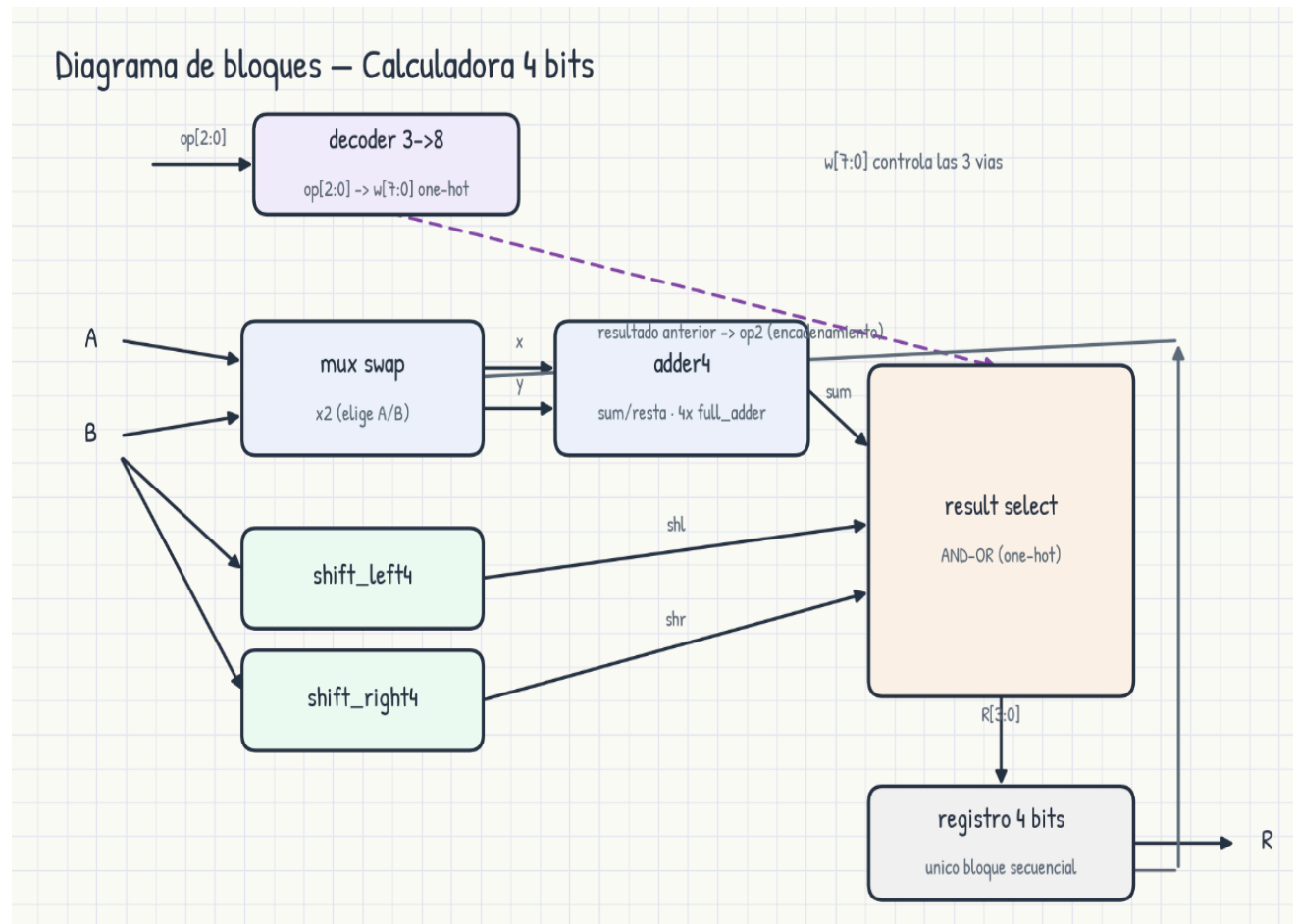
2.1. Decisión de diseño principal: un solo sumador

La alternativa directa sería implementar seis circuitos independientes, uno por operación. Se descartó porque tres de las operaciones son la misma suma con distinta preparación de operandos. El diseño usa un único sumador ripple-carry de 4 bits, reutilizado en las tres operaciones aritméticas, cambiando solamente lo que entra por sus puertos:

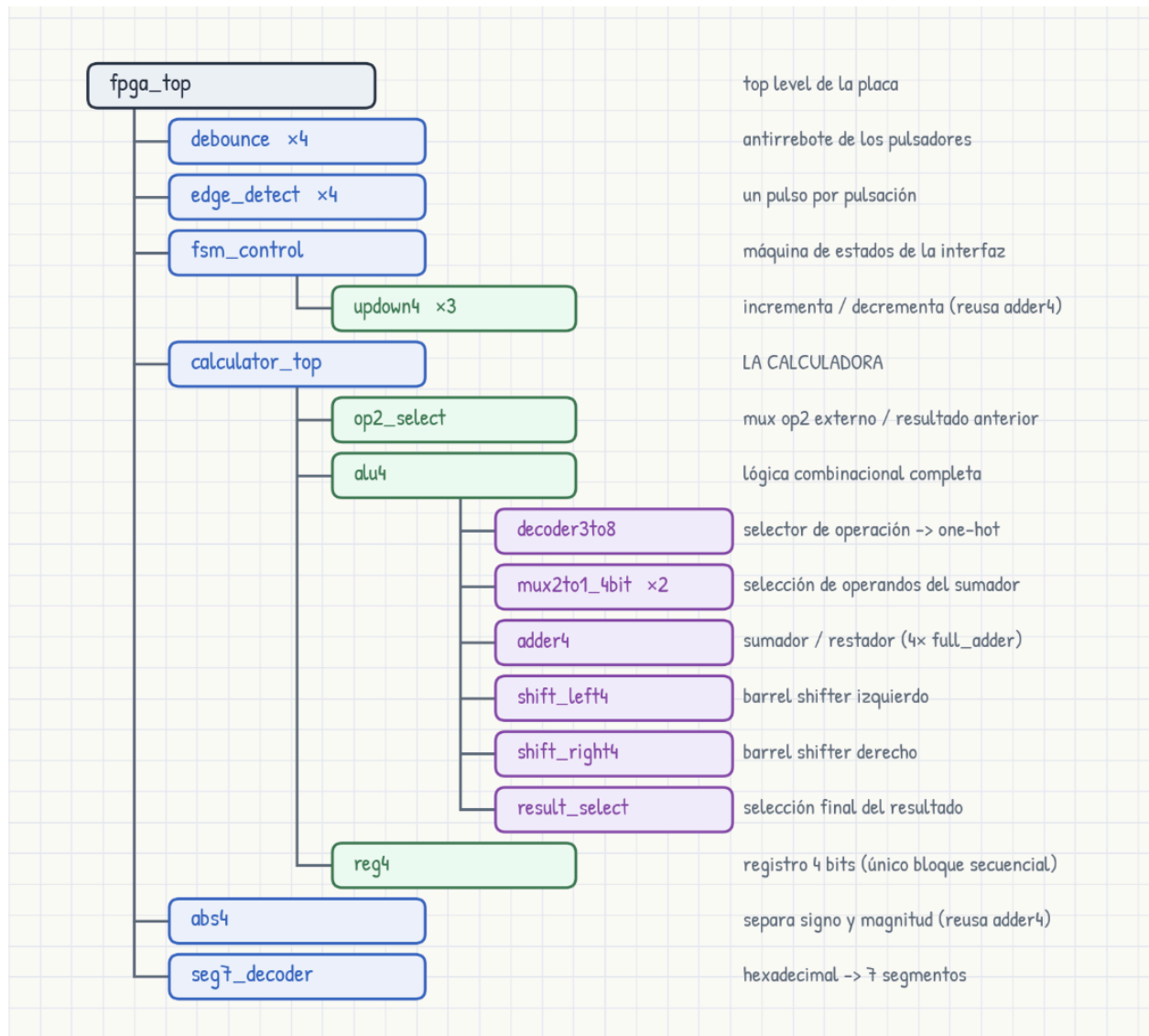
Operación	Código	x	y	sub
Suma	001	A	B	0
Resta	010	A	B	1
Resta inversa	011	B	A	1

Esto reduce el área a aproximadamente un tercio y concentra todo el análisis booleano en un solo bloque de 1 bit (el sumador completo) que después se replica cuatro veces.

2.2. Diagrama de bloques



2.3. Jerarquía de módulos



3. CODIFICACIÓN DE LAS OPERACIONES

Se mantuvo la codificación propuesta en el enunciado.

Código	Operación	Resultado
000	Reinicio	$R = 0000$
001	Suma	$R = A + B$
010	Resta	$R = A - B$
011	Resta inversa	$R = B - A$
100	Shift left	$R = A$ desplazado $B[1:0]$ a la izquierda
101	Shift right	$R = A$ desplazado $B[1:0]$ a la derecha

Los códigos 110 y 111 no se usan. El diseño los trata como shift left y shift right respectivamente, lo que no requiere lógica adicional: simplemente no se decodifican por separado.

Todos los cálculos se realizan en 4 bits. Cuando una operación genera overflow se conservan únicamente los cuatro bits menos significativos, tal como especifica el enunciado.

4. TABLAS DE VERDAD , MAPAS DE KARNAUGH Y EXPRESIONES BOOLEANAS

4.1. Sumador completo de 1 bit

Es el bloque que se replica cuatro veces para formar el sumador de 4 bits, y por lo tanto el análisis más importante del proyecto.

Tabla de verdad					
a b cin	s cout				
0 0 0	0 0				
0 0 1	1 0				
0 1 0	1 0				
0 1 1	0 1				
1 0 0	1 0				
1 0 1	0 1				
1 1 0	0 1				
1 1 1	1 1				

		b cin			
		00	01	11	10
a	0	0	0	0	1
	1	0	1	1	1

$cout = a \cdot b + a \cdot cin + b \cdot cin$

		b cin			
		00	01	11	10
a	0	0	1	1	0
	1	1	0	0	1

$s = a \text{ XOR } b \text{ XOR } cin$

El acarreo de salida corresponde a la función mayoría: cout vale 1 cuando al menos dos de las tres entradas valen 1, lo que da la expresión minimizada $cout = a \cdot b + a \cdot cin + b \cdot cin$.

La suma s, en cambio, forma un patrón de tablero de ajedrez en su mapa: no hay ningún par de unos adyacentes, por lo que la minimización por suma de productos no reduce nada. Ese patrón es la firma característica de la función XOR, así que en lugar de aceptar los cuatro minterminos se implementa directamente como $s = a \text{ XOR } b \text{ XOR } cin$, con dos compuertas XOR en cascada. Este criterio —comprobar si un mapa que no agrupa es en realidad un XOR disfrazado— se aplicó en todo el diseño.

4.2. La resta por complemento a dos

En complemento a dos se cumple que $-y = \text{NOT}(y) + 1$, de donde $x - y = x + \text{NOT}(y) + 1$. Los dos términos adicionales no cuestan hardware extra: el $\text{NOT}(y)$ se obtiene con un XOR usado como inversor controlado ($y_{\text{ef}} = y \text{ XOR } \text{sub}$), que con $\text{sub}=0$ deja pasar y sin cambios y con $\text{sub}=1$ lo invierte bit a bit; y el $+1$ se inyecta por el carry-in de la primera etapa ($\text{cin}_0 = \text{sub}$). Por eso el mismo circuito realiza suma y resta según el valor de una sola señal de control.

Se verifica el circuito con dos casos: una resta de resultado positivo ($5-3$) y otra de resultado negativo ($3-5$).

5 - 3 (resultado positivo)

$x = 0101$ (5)

$y = 0011$ (3)

$\text{sub} = 1 \rightarrow y_{\text{ef}} = 1100, \text{cin}_0 = 1$

etapa 0: $1 + 0 + 1 = 0$, acarreo 1

etapa 1: $0 + 0 + 1 = 1$, acarreo 0

etapa 2: $1 + 1 + 0 = 0$, acarreo 1

etapa 3: $0 + 1 + 1 = 0$, acarreo 1 (se descarta)

$\text{sum} = 0010 = 2$ correcto

3 - 5 (resultado negativo)

$x = 0011$ (3)

$y = 0101$ (5)

$\text{sub} = 1 \rightarrow y_{\text{ef}} = 1010, \text{cin}_0 = 1$

etapa 0: $1 + 0 + 1 = 0$, acarreo 1

etapa 1: $1 + 1 + 0 = 0$, acarreo 1

etapa 2: $0 + 0 + 1 = 1$, acarreo 0

etapa 3: $0 + 1 + 0 = 1$, acarreo 0

$\text{sum} = 1110 = -2$ correcto (complemento a 2)

4.3. Decodificador del selector de operación

Cada salida del decodificador es un mintermino de las tres variables del selector (s_2 s_1 s_0), por lo que no admite minimización: las ocho salidas son independientes y cada una vale 1 para una única combinación del código de operación.

Cada salida = un mintermino de (s_2 s_1 s_0). Codificación one-hot.

w_0	$= s_2' \cdot s_1' \cdot s_0'$	(000)	Reinicio
w_1	$= s_2' \cdot s_1' \cdot s_0$	(001)	Suma
w_2	$= s_2' \cdot s_1 \cdot s_0'$	(010)	Resta
w_3	$= s_2' \cdot s_1 \cdot s_0$	(011)	Resta inv.
w_4	$= s_2 \cdot s_1' \cdot s_0'$	(100)	Shift left
w_5	$= s_2 \cdot s_1' \cdot s_0$	(101)	Shift right
w_6	$= s_2 \cdot s_1 \cdot s_0'$	(110)	(no usada)
w_7	$= s_2 \cdot s_1 \cdot s_0$	(111)	(no usada)

Trabajar con estas señales one-hot es lo que permite que la selección final del resultado (sección 4.4) sea una estructura AND-OR trivial: cada operación activa una sola línea w , y esa línea habilita su vía correspondiente.

4.4. Selección final del resultado

La salida se arma con una estructura AND-OR sobre las señales one-hot del decodificador. Cada vía (suma, shift left, shift right) se habilita con una señal de selección, y un OR final une las tres.

Señales de selección:

$$\text{sel_sum} = w1 + w2 + w3$$

$$\text{sel_shl} = w4 + w6$$

$$\text{sel_shr} = w5 + w7$$

$$r[i] = \text{sum}[i] \cdot \text{sel_sum} + \text{shl}[i] \cdot \text{sel_shl} + \text{shr}[i] \cdot \text{sel_shr}$$

El resultado más notable de esta codificación es que la operación Reinicio no requiere ninguna compuerta adicional: al no seleccionar ninguna vía, todas las AND entregan 0 y el OR final produce 0000 de forma automática.

4.5. Decodificador de 7 segmentos - segmento a

Cuatro variables de entrada (d3 d2 d1 d0) y dieciséis casos. Se desarrolla completo el segmento a; los seis restantes siguen exactamente el mismo procedimiento. El segmento a (la barra superior del display) se enciende en los valores 0, 2, 3, 5, 6, 7, 8, 9, A, C, E, F. Se apaga únicamente en 1, 4, b y d.

		d1 d0				
		00	01	11	10	
d3 d2	00	1	0	1	1	<- 0, 1, 3, 2
	01	0	1	1	1	<- 4, 5, 7, 6
	11	1	0	1	1	<- 12, 13, 15, 14
	10	1	1	0	1	<- 8, 9, 11, 10

Los bordes se tocan: columna 00 ~ 10 y fila 00 ~ 10.

Agrupaciones

Grupo	Celdas	Término
Filas 00+10, columnas 00+10	0, 2, 8, 10	$d_2' \cdot d_0'$
Filas 11+10, columnas 00+10	8, 10, 12, 14	$d_3 \cdot d_0'$
Filas 00+01, columnas 11+10	2, 3, 6, 7	$d_3' \cdot d_1$
Filas 01+11, columnas 11+10	6, 7, 14, 15	$d_2 \cdot d_1$
Fila 01, columnas 01+11	5, 7	$d_3' \cdot d_2 \cdot d_0$
Fila 10, columnas 00+01	8, 9	$d_3 \cdot d_2' \cdot d_1'$

Expresión minimizada:

$$a = d_2' \cdot d_0' + d_3 \cdot d_0' + d_3' \cdot d_1 + d_2 \cdot d_1 + d_3' \cdot d_2 \cdot d_0 + d_3 \cdot d_2' \cdot d_1'$$

Comprobación de minimalidad: las celdas 5 y 9 pertenecen cada una a un único grupo primo, lo que vuelve esenciales los dos últimos términos. Fijados estos, las celdas 0, 3, 12 y 15 quedan también en un único grupo primo cada una, forzando los cuatro primeros términos. Los seis son por lo tanto necesarios y la expresión es mínima.

Verificación en los cuatro casos donde el segmento debe apagarse:

Valor	$d_3 \ d_2 \ d_1 \ d_0$	Resultado
1	0 0 0 1	$a = 0$ correcto
4	0 1 0 0	$a = 0$ correcto
b	1 0 1 1	$a = 0$ correcto
d	1 1 0 1	$a = 0$ correcto

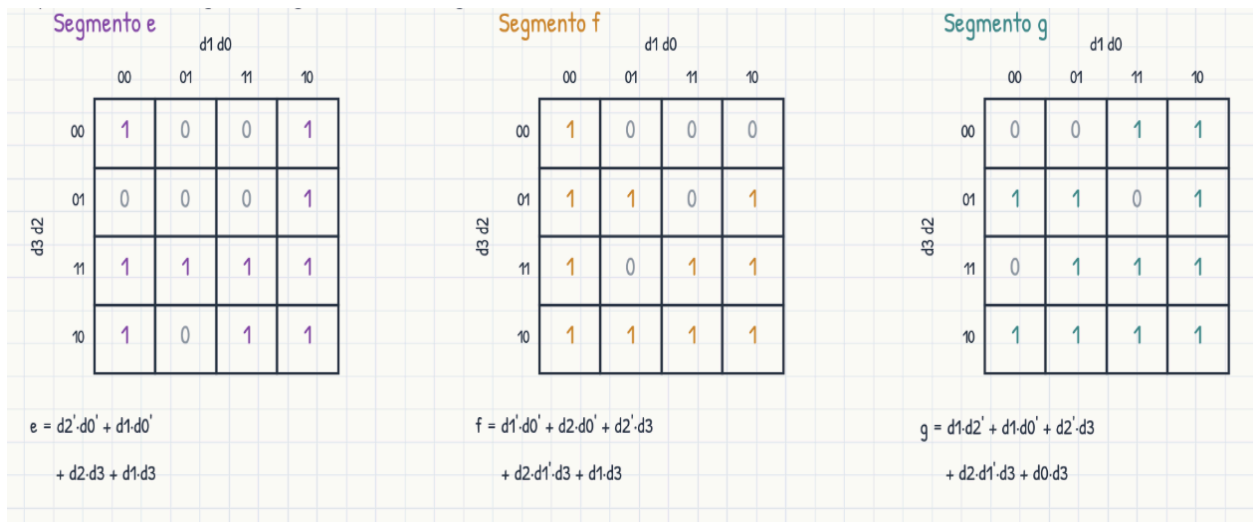
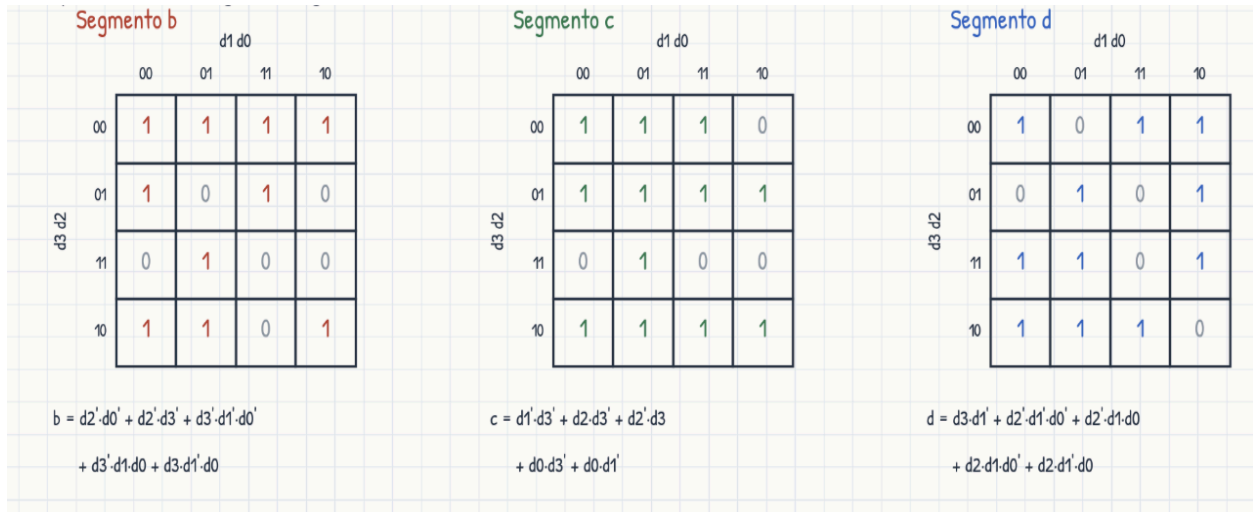
4.6. Segmentos b, c, d, e, f, g

Los seis segmentos restantes se desarrollan con el mismo procedimiento de la sección 4.5. La tabla de verdad completa de los 16 valores se usa como referencia (también está en el encabezado de src/seg7_decoder.v).

Segmentos encendidos por valor, como referencia:

Valor	a	b	c	d	e	f	g
0	1	1	1	1	1	1	0
1	0	1	1	0	0	0	0
2	1	1	0	1	1	0	1
3	1	1	1	1	0	0	1
4	0	1	1	0	0	1	1
5	1	0	1	1	0	1	1
6	1	0	1	1	1	1	1
7	1	1	1	0	0	0	0
8	1	1	1	1	1	1	1
9	1	1	1	1	0	1	1
A	1	1	1	0	1	1	1
b	0	0	1	1	1	1	1
C	1	0	0	1	1	1	0
d	0	1	1	1	1	0	1
E	1	0	0	1	1	1	1
F	1	0	0	0	1	1	1

Mapas de Karnaugh y expresiones minimizadas de cada segmento:



5. IMPLEMENTACIÓN MEDIANTE COMPUERTAS

Toda la lógica combinacional de la calculadora está descrita exclusivamente con primitivas de compuerta de Verilog: and, or, not, xor y buf. No aparece ningún operador de alto nivel de los prohibidos por el enunciado.

5.1. Correspondencia entre expresión booleana y código

Cada expresión booleana obtenida en la sección 4 se traduce directamente a instancias de compuertas. Los dos bloques base son el sumador completo y el multiplexor 2:1; el decodificador y la selección del resultado ya se mostraron en las secciones 4.3 y 4.4.

Sumador completo

```
s  = a XOR b XOR cin
cout = a-b + (a XOR b)-cin

xor u_s1  (a_xor_b, a, b);
xor u_s2  (s,      a_xor_b, cin);
and u_c1  (and_ab, a, b);
and u_c2  (and_pc, a_xor_b, cin);
or  u_cout (cout,  and_ab, and_pc);
```

Multiplexor 2:1 de 1 bit

```
y = a-s' + b-s

not u_ns  (ns,  s);
and u_a_sel (a_sel, a, ns);
and u_b_sel (b_sel, b, s);
or  u_y    (y,   a_sel, b_sel);

Los shifters se arman con estos mismos muxes (2 etapas: x1 y x2).
```

5.2. Los desplazadores

El desplazamiento en sí no cuesta compuertas: es únicamente recablear los bits. Lo único que consume lógica es el multiplexor que decide si el desplazamiento se aplica o no.

Cada desplazador se construye en dos etapas, descomponiendo el monto en binario:

amount	Etapla 1 (×1)	Etapla 2 (×2)	Total
00	no	no	0
01	sí	no	1
10	no	sí	2
11	sí	sí	3

5.3. Sobre el registro de resultado

El registro es lógica secuencial, no combinacional, por lo que no está alcanzado por la restricción del enunciado (que habla explícitamente de "la lógica combinacional de la calculadora"). Aun así se implementó lo más cerca posible del espíritu de la restricción:

- La decisión "mantener el valor / cargar el nuevo resultado" se resuelve con un multiplexor hecho de compuertas, no con un if.
- El reset síncrono se aplica con compuertas AND, no con un if.
- El bloque always contiene únicamente la transferencia del flip-flop.

El operador "`<=`" que aparece en ese bloque es la asignación no bloqueante de Verilog, la forma estándar y correcta de describir un flip-flop, no el operador de comparación "menor o igual" que prohíbe el enunciado.

5.4. Sobre los módulos de interfaz

Los módulos `debounce.v` y `fsm_control.v` corresponden al acondicionamiento de los pulsadores físicos y al secuenciamiento de la interfaz de usuario. No realizan ningún cálculo y son lógica secuencial, por lo que se describen en Verilog conductual, como es estándar para este tipo de lógica. La restricción del enunciado aplica a la lógica combinacional de la calculadora, que está implementada íntegramente con compuertas (`alu4` y sus submódulos, más `seg7_decoder`, `abs4` y `updown4`).

El registro de resultado y los módulos de interfaz son secuenciales por naturaleza —un registro, un contador de antirrebote y una máquina de estados no se implementan con compuertas— y por lo tanto quedan fuera de esa restricción.

6. TESTBENCH, SIMULACIÓN Y GTKWAVE

6.1. Verificación exhaustiva

La ALU se comparó contra un modelo de referencia de alto nivel en las 2048 combinaciones posibles: 16 valores de A × 16 valores de B × 8 operaciones.

-casos probados : 2048
-errores : 0
-RESULTADO : TODO CORRECTO

El modelo de referencia sí utiliza los operadores +, −, << y >>. Esto es válido porque el testbench no se sintetiza: no forma parte del circuito, es únicamente el instrumento de verificación.

6.2. Simulación de las operaciones

Operación	op1	op2	Resultado
Suma	0101 (5)	0011 (3)	1000 (-8)
Resta	0101 (5)	0011 (3)	0010 (2)
Resta Inversa	0101 (5)	0011 (3)	1110 (-2)
Shift Left	0011 (3)	0010 (2)	1100 (-4)
Shift right	1100 (-4)	0010 (2)	0011 (3)
Reinicio	0000 (0)	0000 (0)	0000 (0)

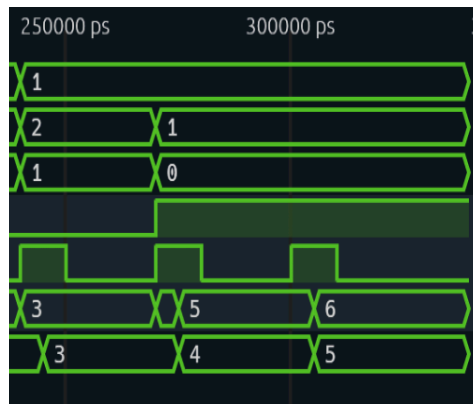


En la forma de onda se observa que alu_out cambia de inmediato (es combinacional) mientras que el resultado solo cambia en el flanco de subida del reloj con ejecutar en alto (es el registro). Ese contraste es la evidencia visual de que el registro es el único bloque secuencial de la calculadora.

6.3. Encadenamiento con el resultado anterior

Con $\text{sel_op2_prev} = 1$, el segundo operando toma el resultado del ciclo previo:

Operación	op1	op2	Resultado
Suma	0010 (2)	0001 (1)	0011 (3)
Suma	0001 (1)	0011 (3) $\leftarrow$ prev	0100 (4)
Suma	0001 (1)	0100 (4) $\leftarrow$ prev	0101 (5)



Se ve cómo el resultado de una operación se reutiliza como operando de la siguiente: $2 + 1 = 3$, luego $1 + 3 = 4$, luego $1 + 4 = 5$.

7. IMPLEMENTACIÓN EN FPGA

7.1. Resultados de síntesis

El diseño se sintetizó con el flujo Yosys → nextpnr-ice40 → icepack → iceprog para la Lattice iCE40 HX1K (package VQ100) de la Nandland Go Board.

Métrica	Valor
Dispositivo	Lattice iCE40 HX1K, package VQ100
Celdas lógicas	204 / 1280 (15 %)
Pines de entrada/salida	23 / 72 (31 %)
Frecuencia máxima	151 MHz (la placa opera a 25 MHz)
Tamaño del bitstream	32.220 bytes

El bajo uso de recursos (15 %) confirma que la reutilización de un único sumador para las tres operaciones aritméticas fue eficiente. La frecuencia máxima está muy por encima de los 25 MHz del reloj de la placa, así que no hay problemas de temporización.

7.2. Interfaz de botones

Botón	Switch	Función
Superior izquierdo	SW1	Incrementar el valor
Inferior izquierdo	SW2	Disminuir el valor
Superior derecho	SW3	Confirmar / ejecutar
Inferior derecho	SW4	Usar el resultado anterior como segundo operando

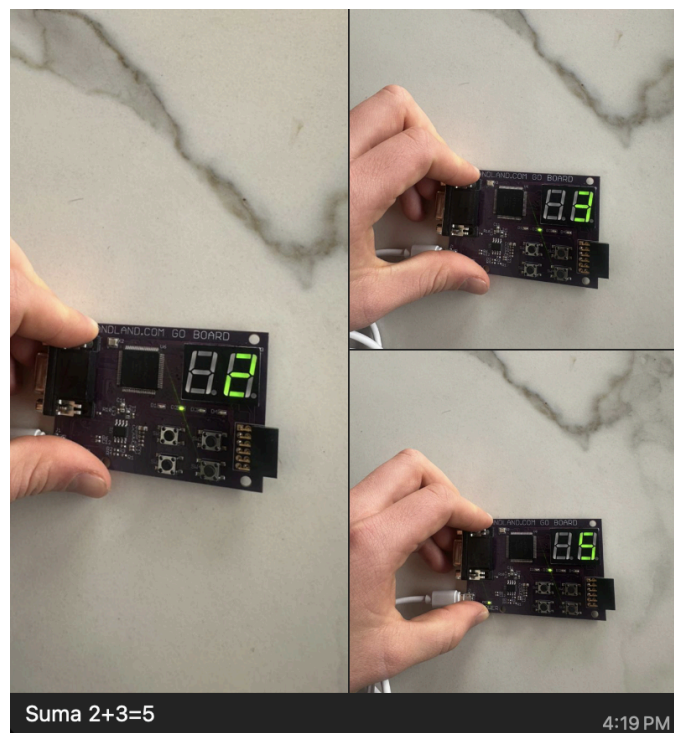
7.3. Secuencia de uso

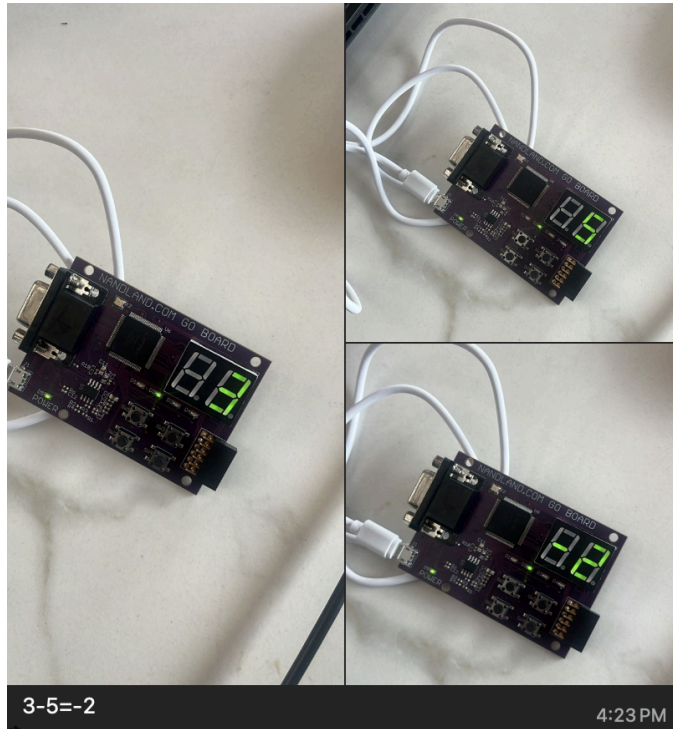
El diseño recorre cuatro etapas, avanzando una con cada pulsación de SW3:

1. **Operación** — el código se muestra en los LED D1, D2 y D3
2. **Primer operando** — se muestra en el display
3. **Segundo operando** — se muestra en el display, o se elige el resultado anterior con SW4 (se enciende D4)
4. **Resultado** — se muestra en el display; al confirmar de nuevo se vuelve a la etapa 1

El display izquierdo muestra el signo y el derecho la magnitud del resultado (por ejemplo, -2 se muestra como "-2", no como su hexadecimal crudo). Esto sigue la indicación del profesor de mostrar el valor con signo y magnitud.

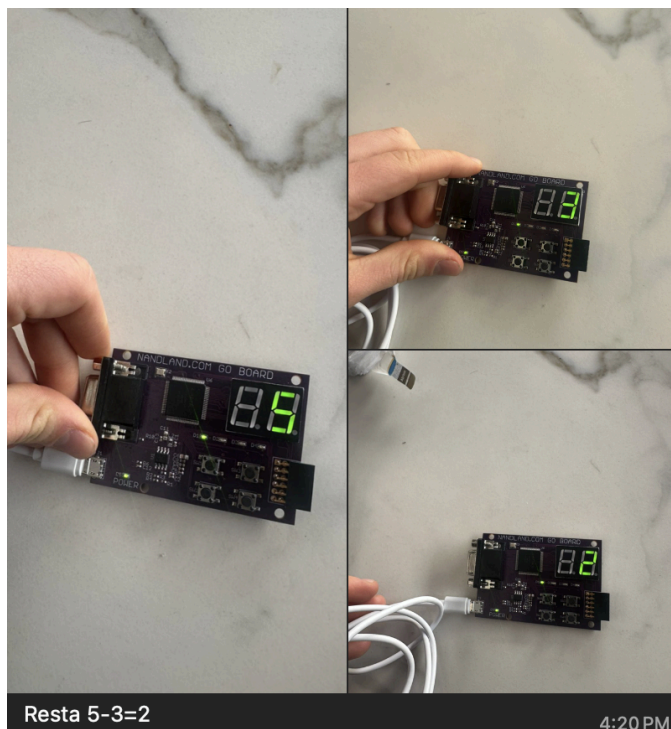
A continuación se documenta cada operación con su secuencia completa en la Go Board (ingreso de op1, ingreso de op2 y resultado), demostrando el funcionamiento del diseño sobre la FPGA:





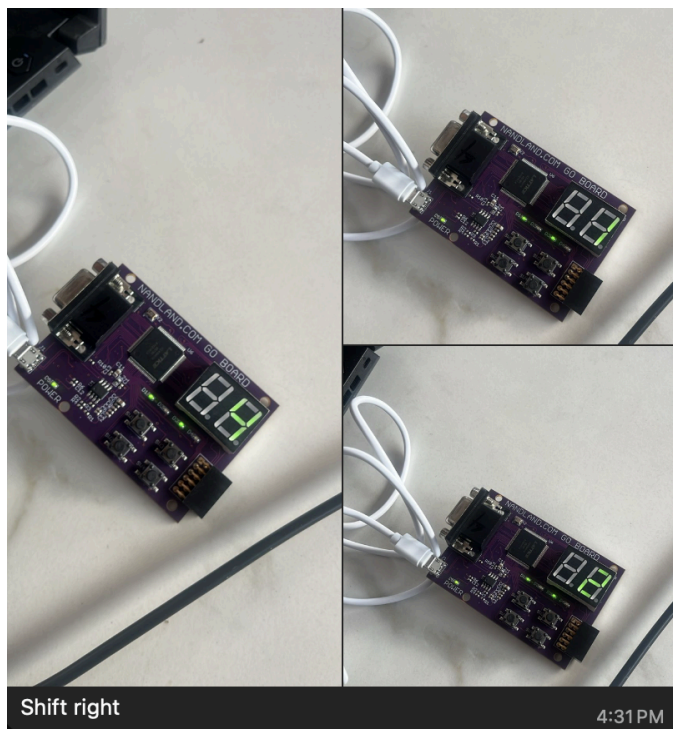
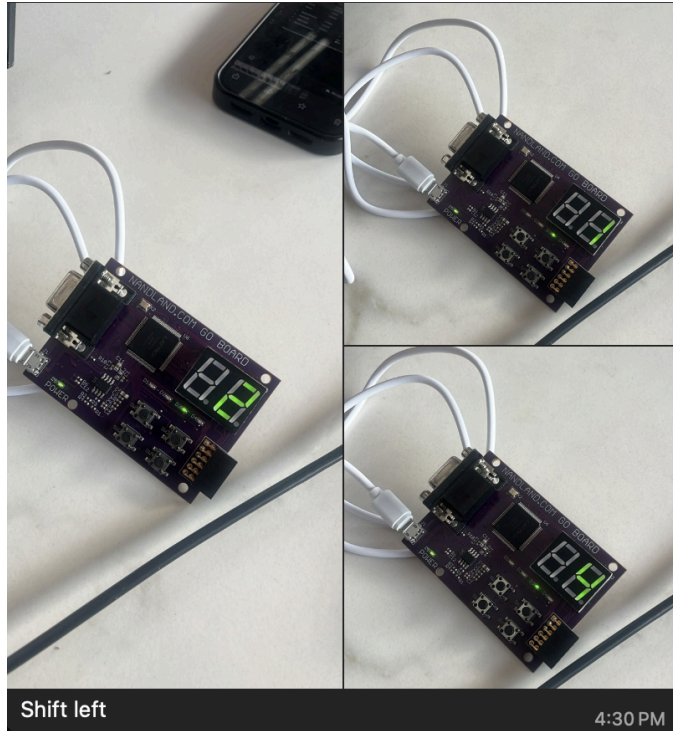
3-5=-2

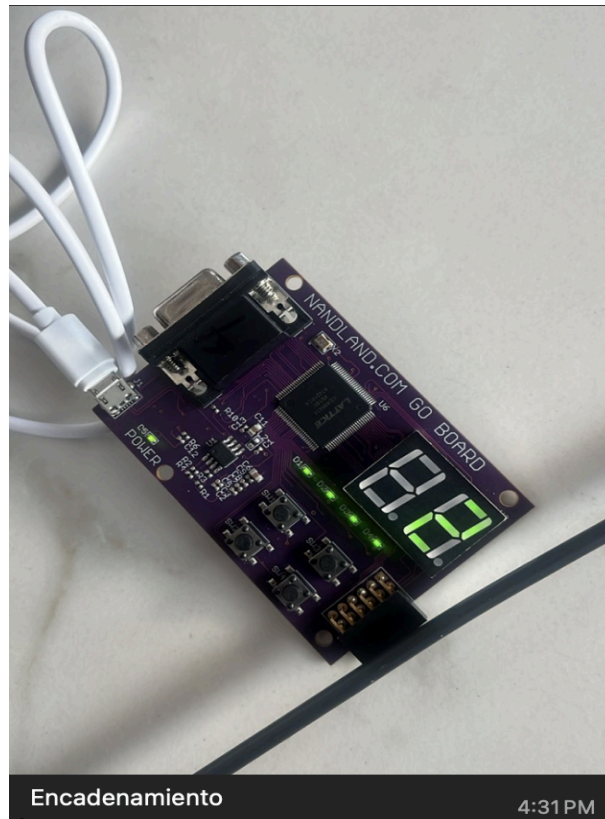
4:23 PM



Resta 5-3=2

4:20 PM





El LED D4 encendido indica que el segundo operando corresponde al resultado de la operación previa, verificando la capacidad de encadenar operaciones.

En todos los casos, el resultado mostrado en la placa coincidió con el valor esperado para la operación y los operandos ingresados, de forma consistente con el comportamiento verificado en la simulación (sección 6). Esto confirma que el diseño funciona correctamente sobre la FPGA.

8. CONCLUSIONES

Reutilizar un único sumador ripple-carry de 4 bits para las tres operaciones aritméticas (suma, resta y resta inversa) redujo el área a aproximadamente un tercio y, más importante, concentró todo el análisis booleano en un solo bloque de 1 bit —el sumador completo— que luego se replica cuatro veces. El esfuerzo de diseño se invierte una vez y se reaprovecha.

La representación en complemento a dos resultó especialmente elegante de implementar a nivel de compuertas. Los dos términos que la definen, $-y = \text{NOT}(y) + 1$, no cuestan hardware extra: el $\text{NOT}(y)$ se obtiene con un XOR usado como inversor controlado ($y_{\text{ef}} = y \text{ XOR sub}$) y el $+1$ se inyecta por el carry-in de la primera etapa ($\text{cin}_0 = \text{sub}$). Así, un mismo circuito hace suma y resta según una sola señal de control.

El hallazgo más ilustrativo fue que la operación de Reinicio no requiere ninguna compuerta adicional: al usar codificación one-hot, el código 000 activa una señal (w_0) que no participa en ninguna vía de selección, de modo que todas las vías entregan 0 y el OR final produce 0000. El reinicio es la consecuencia estructural de no seleccionar ninguna vía, no un circuito construido aparte.

Las principales dificultades fueron de la interfaz física, no del cálculo: el rebote mecánico de los pulsadores (resuelto con el módulo debounce), la polaridad de los displays de ánodo común y el manejo del overflow al conservar sólo los cuatro bits menos significativos. La verificación exhaustiva de las 2048 combinaciones posibles contra un modelo de referencia (0 errores) dio confianza de que la lógica combinacional es correcta antes de llevarla a la placa.

Anexo: método para completar los mapas de Karnaugh

Para cada uno de los segmentos se siguió el mismo procedimiento:

1. Escribir los 16 valores de la tabla de verdad (referencia en la sección 4.6).
2. Dibujar el mapa de 4×4 con las filas d3 d2 y las columnas d1 d0 en orden Gray (00, 01, 11, 10). Ese orden es lo que hace que dos celdas vecinas difieran en un solo bit.
3. Agrupar los unos en rectángulos de 1, 2, 4 u 8 celdas, buscando siempre los grupos más grandes posibles, y recordando que los bordes del mapa se tocan.
4. Cada grupo aporta un término al resultado: las variables que no cambian dentro de ese grupo.
5. Comprobar la expresión obtenida en los casos donde el segmento debe estar apagado. Es ahí donde se detectan los errores de agrupación.

Un criterio útil: si el mapa queda con patrón de tablero de ajedrez y no agrupa nada, la función es un XOR y conviene implementarla como tal en lugar de aceptar la forma canónica.